

Cloud Deployment Report

Two AWS EC2 free-tier VMs, a managed Aiven PostgreSQL database, and everything that broke — and got fixed — along the way.

● 10/10 stations running

● central-station healthy

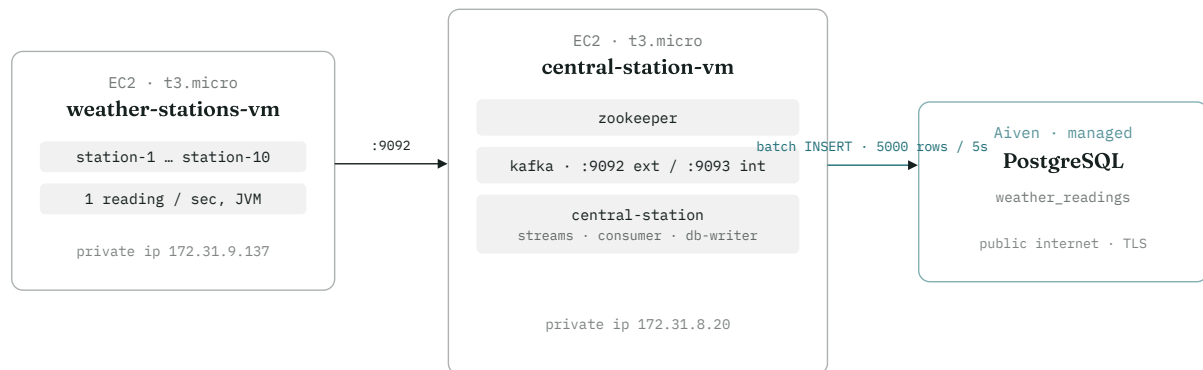
● Kafka + Zookeeper healthy

● Aiven Postgres connected

This session took the Weather Stations Monitoring system — already implemented and tested locally with Docker Compose and Kubernetes manifests — and deployed it as the assignment's **cloud bonus**: the ten weather stations and the Kafka/central-station stack split across two separate AWS virtual machines, writing into a PostgreSQL database managed by Aiven rather than a self-hosted one. Everything below reflects the system as it stood at the end of the session, with real data pulled live from the deployment.

01 Architecture

Same application code as the Kubernetes deliverable — the only thing that changes for the bonus is where each piece runs.



Stations reach Kafka's *external* listener over the shared VPC's private network; central-station talks to Kafka's *internal* listener on the same host, then batches writes out to Aiven over the public internet with TLS.

Full technical detail on this split (listener configuration, security groups, compose files) lives in `c1oud/README.md` in the repository.

02 Deployment log

What was actually clicked and run, in order.

● Provisioned two EC2 instances

us-east-1 (N. Virginia) · Ubuntu Server 22.04 LTS · t3.micro · same VPC + subnet
weather-stations-vm and central-station-vm, one shared key pair, placed in the same subnet so they can reach each other over private IPs without exposing anything extra to the internet.

● Locked down security groups

SSH (22) restricted to my IP on both. Kafka (9092) on central-station-vm opened only to weather-stations-sg — not 0.0.0.0/0 — so the broker is reachable by the other VM and nothing else.

● Created the Aiven PostgreSQL service

Free tier, SSL enforced by default. Connection details (host, port, database, user, password) captured and injected only as environment variables on the VM — never committed to the repo.

● Bootstrapped Docker on both VMs

Installed Docker CE + the Compose plugin via `cloud/bootstrap.sh`, cloned the repository onto each machine.

● Brought up the central stack, then the stations

`docker compose -f docker-compose.central.yml up -d --build` on the central VM, then the equivalent for the ten stations once Kafka was confirmed healthy.

● Verified end to end

Watched readings land in Aiven, rain alerts appear on the Kafka topic, and per-station counts settle into a stable, balanced pattern.

03 What went wrong, and the fix

Free-tier VMs have 1 GB of RAM. Default JVM heap sizing does not know that.

INCIDENT

Kafka crash-looping on central-station-vm

- Symptom** Kafka container stuck restarting; `docker compose ps` showed `Restarting (1)` in a loop.
- Cause** Bitnami's Kafka image defaults to a 1 GB JVM heap — that alone exceeds the VM's entire 908 MB of RAM, before Zookeeper or central-station even start.
- Fix** `KAFKA_HEAP_OPTS=-Xmx350m`, `ZOO_HEAP_SIZE=160`, `JAVA_TOOL_OPTIONS=-Xmx224m` for central-station, plus a 512 MB swap file for headroom.

INCIDENT

Disk full on the same VM

- Symptom** `fallocate: No space left on device` — the 8 GB default root volume was completely full.
- Cause** Docker images, Maven build cache, and (ironically) the swap file added moments earlier all competed for the same 8 GB disk.
- Fix** Pruned the Docker build cache, cleaned `apt`'s cache, and right-sized the swap file to 512 MB instead of the 2 GB first attempted.

INCIDENT

Weather-stations VM thrashing

- Symptom** Load average of **44** on a 2-vCPU box; 27 MB RAM free; SSH sessions barely responding.
- Cause** Ten separate JVMs, each defaulting to roughly a quarter of host RAM (~227 MB) with no cap set — ten stations tried to claim ~2.3 GB combined on a 908 MB machine.
- Fix** `JAVA_TOOL_OPTIONS=-Xmx48m` per station — plenty for an app sending one small JSON message a second — plus a 1 GB swap file as a safety margin. Load dropped from 44 to under 6 within a minute of recreating the containers.

All three fixes are committed to `cloud/docker-compose.central.yml` and `cloud/docker-compose.stations.yml` on main, so the deployment reproduces cleanly for anyone following the same steps.

04 Evidence

Captured directly from the running deployment.

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, the navigation bar shows the AWS logo, a search icon, and the user's name 'Zeyad Essam (361960319289)' in the 'United States (N. Virginia)' region. The main header indicates 'EC2 > Instances'. Below this, the 'Instances (1/2)' section shows a table of running instances. The first instance, 'ions-vm', is highlighted, and its details are expanded below. The details panel shows the instance ID 'i-0fb5507c6cf220a3d', its public IPv4 address '44.203.225.164', private IPv4 address '172.31.9.137', and public DNS 'ec2-44-203-225-164.compute-1.amazonaws.com'. A tooltip indicates the public IPv4 address has been copied to the clipboard. The second instance, 'central-station-vm', is also running and its details are partially visible.

Name	Instance ID	Instance state	Instance type	Status check
ions-vm	i-0fb5507c6cf220a3d	Running	t3.micro	3/3 checks pas
central-station...	i-05d324b791327be8f	Running	t3.micro	Initializing

i-0fb5507c6cf220a3d (weather-stations-vm)

▼ Instance summary Info

Instance ID i-0fb5507c6cf220a3d	Public IPv4 address 44.203.225.164 open address Copy public IPv4 address to clipboard	Private IPv4 addresses 172.31.9.137
IPv6 address -	Instance state Running	Public DNS ec2-44-203-225-164.compute-1.amazonaws.com open address

Both EC2 instances running. weather-stations-vm and central-station-vm, both t3.micro in us-east-1, status checks passing.


```

ubuntu@ip-172-31-8-20: ~/weather-lab/cloud
ation_id":3,"humidity":76,"status_timestamp":1787567138,"message":"Raining at station 3"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":8,"humidity":74,"status_timestamp":1787567139,"message":"Raining at station 8"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":2,"humidity":85,"status_timestamp":1787567139,"message":"Raining at station 2"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":3,"humidity":78,"status_timestamp":1787567139,"message":"Raining at station 3"}
[weather-readings-consumer] INFO com.netcentric.centralstation.db.DbWriter - Persisted batch of 48 readi
ngs
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":5,"humidity":93,"status_timestamp":1787567140,"message":"Raining at station 5"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":8,"humidity":83,"status_timestamp":1787567140,"message":"Raining at station 8"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":3,"humidity":97,"status_timestamp":1787567140,"message":"Raining at station 3"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":9,"humidity":83,"status_timestamp":1787567141,"message":"Raining at station 9"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":8,"humidity":78,"status_timestamp":1787567141,"message":"Raining at station 8"}
[rain-alerts-consumer] INFO com.netcentric.centralstation.consumer.RainAlertsConsumer - RAIN ALERT: {"st
ation_id":4,"humidity":73,"status_timestamp":1787567141,"message":"Raining at station 4"}
ubuntu@ip-172-31-8-20:~/weather-lab/cloud$ ubuntu@ip-172-31-9-137:~/weather-lab/cloud$ docker exec -it c
loud-kafka-1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9093 --topic rain-alerts --from-beginning
Error response from daemon: No such container: cloud-kafka-1
ubuntu@ip-172-31-9-137:~/weather-lab/cloud^C
ubuntu@ip-172-31-8-20:~/weather-lab/cloud$ docker exec -it cloud-kafka-1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9093 --topic rain-alerts --max-messages 10
{"station_id":1,"humidity":97,"status_timestamp":1787567195,"message":"Raining at station 1"}
{"station_id":5,"humidity":76,"status_timestamp":1787567195,"message":"Raining at station 5"}
{"station_id":7,"humidity":77,"status_timestamp":1787567195,"message":"Raining at station 7"}
{"station_id":1,"humidity":84,"status_timestamp":1787567196,"message":"Raining at station 1"}
{"station_id":5,"humidity":92,"status_timestamp":1787567196,"message":"Raining at station 5"}
{"station_id":9,"humidity":99,"status_timestamp":1787567196,"message":"Raining at station 9"}
{"station_id":3,"humidity":90,"status_timestamp":1787567196,"message":"Raining at station 3"}
{"station_id":1,"humidity":71,"status_timestamp":1787567197,"message":"Raining at station 1"}
{"station_id":5,"humidity":88,"status_timestamp":1787567197,"message":"Raining at station 5"}
{"station_id":8,"humidity":100,"status_timestamp":1787567197,"message":"Raining at station 8"}
Processed a total of 10 messages
ubuntu@ip-172-31-8-20:~/weather-lab/cloud$

```

Rain-alerts topic, read directly. `kafka-console-consumer.sh --topic rain-alerts` on `central-station-vm`, confirming the Kafka Streams processor is correctly filtering `humidity > 70%` and republishing alerts.

```

ubuntu@ip-172-31-9-137: ~/weather-lab/cloud

ubuntu@ip-172-31-9-137:~/weather-lab/cloud$ ^[[200~docker run --rm -it postgres:16-alpine psql \
> 'host=weather-station-db-ammaraInahas9-dec4.h.aivencloud.com port=10545 dbname=defaultdb user=avnadmin pa
ssword=AVNS_bt5gQ32Pqz56aR8Gjrk sslmode=require'
docker: command not found
ubuntu@ip-172-31-9-137:~/weather-lab/cloud$ docker run --rm -it postgres:16-alpine p
sql 'host=weather-station-db-ammaraInahas9-dec4.h.aivencloud.com port=10545 dbname
=defaultdb user=avnadmin password=AVNS_bt5gQ32Pqz56aR8Gjrk sslmode=require'
psql (16.15, server 18.6)
WARNING: psql major version 16, server major version 18.
         Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

defaultdb=> SELECT * FROM weather_readings LIMIT 20;
 id | station_id | sequence_number | battery_status | timestamp      | humidity | temperature | wind_speed
-----+-----+-----+-----+-----+-----+-----+-----
  1 |          8 |              1 | low            | 1787564190     |      98  |          37  |          41
  2 |          3 |              1 | low            | 1787564190     |      91  |          52  |          58
  3 |          4 |              1 | low            | 1787564190     |      49  |          71  |          44
  4 |          4 |              2 | high           | 1787564199     |      73  |          69  |          24
  5 |          4 |              3 | medium         | 1787564200     |      52  |          97  |          51
  6 |          4 |              4 | medium         | 1787564201     |      44  |         120  |          26
  7 |          4 |              5 | high           | 1787564202     |      75  |          69  |          42
  8 |          4 |              6 | medium         | 1787564204     |      64  |         112  |          39
  9 |          8 |              2 | medium         | 1787564198     |      45  |          52  |          28
 10 |          8 |              4 | low            | 1787564201     |      62  |          81  |          55
 11 |          8 |              5 | medium         | 1787564202     |      39  |          88  |          16
 12 |          8 |              6 | low            | 1787564203     |      93  |          33  |           8
 13 |          3 |              2 | medium         | 1787564198     |      39  |          30  |          31
 14 |          3 |              3 | medium         | 1787564200     |      22  |          68  |           3
 15 |          3 |              4 | high           | 1787564201     |      84  |         115  |          42
 16 |          3 |              5 | low            | 1787564202     |      81  |         109  |           7
 17 |          3 |              6 | high           | 1787564203     |      41  |          84  |          14
 21 |          59 |              7 | medium         | 1787564204     |      64  |         --More--
 19 |          7 |              1 | low            | 1787564190     |      41  |          20  |          56
 20 |          7 |              2 | medium         | 1787564198     |      74  |          60  |          55
(20 rows)

```

Raw rows in Aiven PostgreSQL. Connected straight to weather-station-db-ammaraInahas9-dec4.h.aivencloud.com — a database this session never touched except over the network, exactly as the bonus requires.

ubuntu@ip-172-31-9-137: ~/weather-lab/cloud

psql (16.15, server 18.6)

WARNING: psql major version 16, server major version 18.

Some psql features might not work.

SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)

Type "help" for help.

defaultdb=> SELECT * FROM weather_readings LIMIT 20;

id	station_id	sequence_number	battery_status	timestamp	humidity	temperature	wind_speed
1	8	1	low	1787564190	98	37	41
2	3	1	low	1787564190	91	52	58
3	4	1	low	1787564190	49	71	44
4	4	2	high	1787564199	73	69	24
5	4	3	medium	1787564200	52	97	51
6	4	4	medium	1787564201	44	120	26
7	4	5	high	1787564202	75	69	42
8	4	6	medium	1787564204	64	112	39
9	8	2	medium	1787564198	45	52	28
10	8	4	low	1787564201	62	81	55
11	8	5	medium	1787564202	39	88	16
12	8	6	low	1787564203	93	33	8
13	3	2	medium	1787564198	39	30	31
14	3	3	medium	1787564200	22	68	3
15	3	4	high	1787564201	84	115	42
16	3	5	low	1787564202	81	109	7
17	3	6	high	1787564203	41	84	14
21	59	7	medium	1787564204	64	--More--	
19	7	1	low	1787564190	41	20	56
20	7	2	medium	1787564198	74	60	55

(20 rows)

defaultdb=> SELECT battery_status, COUNT(*) FROM weather_readings GROUP BY battery_status;

battery_status	count
medium	8187
high	6196
low	6133

(3 rows)

Battery distribution query, live. The same analysis required in Section E of the assignment, run straight against the cloud deployment's data.

05 Required analysis, from this deployment

Same two queries as the local/Kubernetes run — this time against the live Aiven database.

Battery status distribution (target: 30% low / 40% medium / 30% high) — 20,516 readings collected during this session:

BATTERY_STATUS	COUNT	SHARE
medium	8,187	39.9%
high	6,196	30.2%
low	6,133	29.9%

Within a few tenths of a percent of the spec on all three bands.

Per-station reading counts, sampled mid-run — evidence of ~10% simulated message loss and balanced production across all ten stations:

Sample taken ~28 minutes into the run, before the final counts above

STATION_ID	READINGS
1	1,776
2	1,758
3	1,748
4	1,780
5	1,758
6	1,745
7	1,757
8	1,729
9	1,748
10	1,759

The full dropped-message query (`MAX(sequence_number) - COUNT(*)` per station) is in `sql/analysis_queries.sql` — run it against the Aiven database with the connection command in the checklist below to get final per-station drop rates for the report.

06 Current state & next steps

What's live right now, and what's left before this can be called finished.

2

EC2 VMS RUNNING

10

STATION
CONTAINERS

1

AIVEN POSTGRES
SERVICE

20.5k+

ROWS PERSISTED

Both EC2 instances and the Aiven service are still running. They're within AWS/Aiven free-tier limits for now, but should be torn down once evidence-gathering is done, to avoid any risk of charges accruing.

- ☒ Weather station producer implemented, tested, and running on real infrastructure
- ☒ Kafka Streams rain-detection topology verified against the live `rain-alerts` topic
- ☒ Central station batch-writing confirmed against an externally managed database
- ☒ Battery-status distribution analysis run against cloud data (39.9 / 30.2 / 29.9%)
- ☐ Run the dropped-messages query from `sql/analysis_queries.sql` for final per-station numbers
- ☐ Assemble this report plus the Kubernetes deployment evidence into the final submission PDF
- ☐ Terminate both EC2 instances and delete the Aiven service once finished

To pull the final drop-rate numbers, from either VM:

```
docker run --rm postgres:16-alpine psql 'host=weather-station-db-ammaraInahas9-d
ec4.h.aivencloud.com port=10545 dbname=defaultdb user=avnadmin sslmode=require'
-f sql/analysis_queries.sql
```

Weather Stations Monitoring — Lab 4, Cloud Deployment (Bonus)

github.com/AmmarAlnahas99/Weather_Station_Final_Lab